IR HomeWork-3 Report

Team Members: Abdulaziz Arif Suria, Ekam Chahal, Jai Surya Kode, Saumya Gupta

1. **Crawling [Note down steps implemented for each of the below]**

   a. **URL Canonicalization –**We followed the required 5 steps as follows:
      - Parse the URL using urllib python module
      - Converting upper case to lowercase
      - Removing ports
      - Making relative urls absolute
      - Removing text after "#" in between a link i.e. fragment removal
      - Removing duplicate '//' except when its after https:

      We have created a function **canonicalizeURL** which performs all these tasks on a given URL

   b. **Frontier Management –**
      We have adopted a priority queue with focus on 3 different criterions.
      1. Wave number: Based on BFS Traversal
      2. URL score: if URL contains words from a predefined set of keywords, it is score higher
      3. Inlinks count: If URL has higher inlinks count, it is score higher.

      Formulation for the scoring criteria:

      KEYWORDS=['accident','chernobyl','kyshtym','disaster','fukushima','nuclear','cancer','radioactive','environment','radiation','contamination','incident','evacuation','gov','org','long-term','safety']

      FORMULATION: e^(-KEYWORD_HITS) + e^(-INLINKS_COUNT)
      (-ve sign as python follows a minheap by default)

      We have also used multi-threading using the concurrent.ThreadPoolExecutor and thread locking.

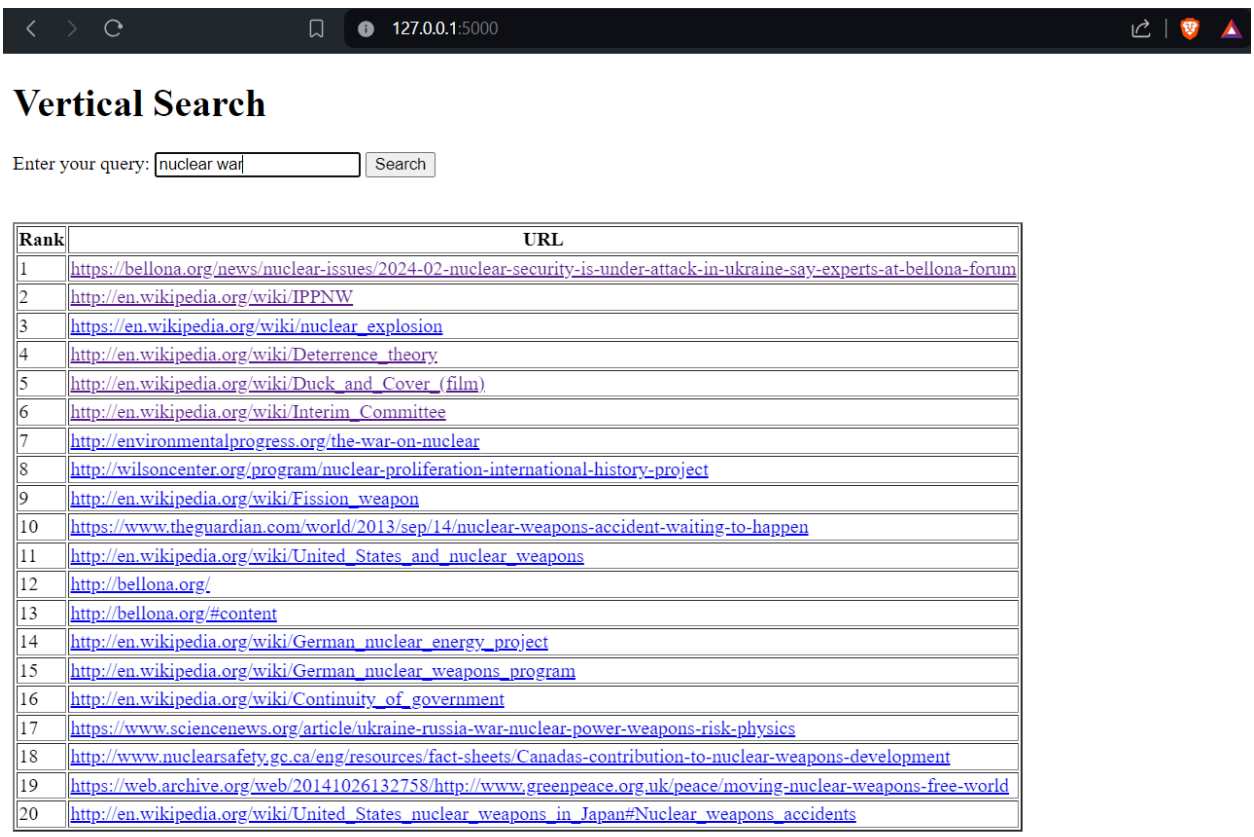
   c. **Politeness Policy –** For following this we ensured that we only make 1 request to each url and we have a 1 second delay between each request we make. We also fetch the robots.txt for a page and obey their required policy for fetching.

   d. **Document Processing –** The HTML text is processed using bs4 library to extract all tags, which produces the links that appear on the page. For consistency, canonicalization is applied to every URL and the adds to the frontier for each URL, we also add new URLs to a list and determine whether or not they have been

alredy visited and we save the out links to a file. Using bs4, the content of the document is cleaned up of all HTML formatting, scripts, and styles. It is then saved to a file, with the canonical URL as the DOCNO, the content of the title tag in head tag. For the stored web data processing while merging, we have reused the code for document processing which we used for the previous assignments. We take the tags and match the tags and retrieve the text and store it in a dictionary.

2. Vertical Search

a. Add a Screenshot of your Vertical Search UI



Rank	URL
1	https://bellona.org/news/nuclear-issues/2024-02-nuclear-security-is-under-attack-in-ukraine-say-experts-at-bellona-forum
2	http://en.wikipedia.org/wiki/IPPNW
3	https://en.wikipedia.org/wiki/nuclear_explosion
4	http://en.wikipedia.org/wiki/Deterrence_theory
5	http://en.wikipedia.org/wiki/Duck_and_Cover_(film)
6	http://en.wikipedia.org/wiki/Interim_Committee
7	http://environmentalprogress.org/the-war-on-nuclear
8	http://wilsoncenter.org/program/nuclear-proliferation-international-history-project
9	http://en.wikipedia.org/wiki/Fission_weapon
10	https://www.theguardian.com/world/2013/sep/14/nuclear-weapons-accident-waiting-to-happen
11	http://en.wikipedia.org/wiki/United_States_and_nuclear_weapons
12	http://bellona.org/
13	http://bellona.org/#content
14	http://en.wikipedia.org/wiki/German_nuclear_energy_project
15	http://en.wikipedia.org/wiki/German_nuclear_weapons_program
16	http://en.wikipedia.org/wiki/Continuity_of_government
17	https://www.sciencenews.org/article/ukraine-russia-war-nuclear-power-weapons-risk-physics
18	http://www.nuclearsafety.gc.ca/eng/resources/fact-sheets/Canadas-contribution-to-nuclear-weapons-development
19	https://web.archive.org/web/20141026132758/http://www.greenpeace.org.uk/peace/moving-nuclear-weapons-free-world
20	http://en.wikipedia.org/wiki/United_States_nuclear_weapons_in_Japan#Nuclear_weapons_accidents

b. Explain briefly how you implemented it.

We have utilized Flask as our backend and utilized the ES-builtin Module as our standard retrieval model. The frontend is displayed using a simple HTML and a table format with the help of JINJA.

When user searches the query, a post request is made to the backend via the HTML form which further calls the ES-Builtin function on the user search query and returns the results as a table consisting of the RANK and the URL as required.

3. **Extra Credits Done [Note done what was done for each extra credit]**
   Speed Optimization:
   We have added a default timeout of 2 seconds on request.get() method which allows us to save time for slow-responding URLs and helps us to crawl pages effectively. We didn't have to modify our structure and this improved the speed optimally as few pages took as long as 10 seconds to respond.